

실시간 API 데이터를 활용한 싱크홀 위험도 지도 (가제목)

<최종 결과물 예상>

1. 실시간 API 기반 싱크홀 위험도 지도 (+재난 알림 문자)
2. 실제 시뮬레이션용 랜덤값 싱크홀 위험도 지도 (+재난 알림 문자)

싱크홀 위험도 구성도(서울시 500mX500m 격자, 5분 간격)

- **1단계(초록):** 지반액상화+서울시 누수관 정도+여러 지반 탐사 지도 정보 기반
- **2단계(노랑):** 1단계+1시간 강수량, 교통량, 지하수위 변화량에 따라 격상
- **3단계(빨강):** 누적단계 -> 2단계+지하수위 급변+실시간교통량(수정 예정)

*참고: 스키마에 있는 score 컬럼을 예를 들어 "정적 기저점수(0~60) + 실시간 가중치(0~40)"처럼 나누면, 왜 1단계가 정해졌고 왜 2·3단계로 격상되는지 근거를 분리해서 설명하기 쉬움. score계산식(가중치 배분) 설계 필요시 진행.

(정확한 기준 정하기)

< 1단계(배경) 위험도 판별 요소 4가지 >

1단계 배경 레이어 4종 (체크리스트와 매칭)		
레이어	출처	비고
시추공 지층/N값	국토지반정보포털(geoinfo.or.kr), 서울시 시추공 공간정보(data.go.kr)	N값 낮을수록 무른 지반
하수관 노후도·누수	한국환경공단_하수도정보(data.go.kr), 서울 열린데이터광장 "하수관거 통계"	자치구별 30/50년 이상 비율
액상화 위험등급	서울시 3D 액상화위험지도(Virtual Seoul/S-Map), 관련 언론보도	등급만 비공개 지도에서 유추, 원자료는 비공개라 뉴스 기반 정리 필요
과거 침하/공동 이력	지하안전정보시스템(jis.go.kr) 지반침하사 고DB	건수 자체는 적음(보조지표)

*지반 정보는 정부 공식 기준 활용 - 「지하안전관리에 관한 특별법」 및 시행령 참고.

< 2단계 격상 판별 요소 >

• 호우 단계별 정부 기준

- 호우주의보: 3시간 60mm 이상 또는 12시간 110mm 이상
- 호우경보: 3시간 90mm 이상 또는 12시간 180mm 이상
- 극한호우(최고 단계): 1시간 누적 50mm 이상이면서 3시간 누적 90mm 이상이거나, 1시간 누적 72mm 이상인 경우 — 긴급재난문자 발송.

< 3단계 격상 판별 요소 - "누적 조건" >

* 2단계에 누적으로 지하수위급변을 확인하는 이유: 어쩌다 혼자 지하수위가 튼 경우 방지하여 호우로 인한 지하수위 급변만 확인하기 위해서

1) 지하수위 변화도

: 지하수위 급강하는 전국 표준 수치 기준은 없음.

- 실무적 대안: 절대값 대신 관측정별 평년(baseline) 대비 상대적 변동폭을 기준으로 잡는 방법을 제안.
- 예시: 최근 30일 평균 대비 표준편차(σ) 이상 급락 시 2단계, 2σ 이상이면 3단계 — 이렇게 하면 "이 지역만의 정상 범위"를 기준으로 삼아 지역마다 지반이 다른 문제를 피할 수 있다. 이 방식은 "근거"로 남기기에 적절 (공식 기준 부재를 명시하고 대안 설계 이유를 적으면 됨).

2) 실시간 교통량(교통혼잡도)

어케 할지 못했음.....

<필요한 데이터>

1. 실시간 강수량(기상청 API허브(apihub.kma.go.kr))

: 500m 해상도의 고해상도 격자분석데이터를 5분 주기로 NetCDF4 포맷으로 실시간 제공하며, 강수량 등 요소를 포함)

- 기상청 AI허브 사용법 및 고해상도 지도 활용법.pdf
(아래 링크 들어가면 pdf 파일 2개 있음)

<https://research.kookmin.ac.kr/information/promotion/427?pn=46rssrssr>



2. 실시간 교통량 API: 서울특별시_실시간 도로 소통 정보 (data.go.kr / data.seoul.go.kr)

-> 일반도로는 서울시 카드택시 7만여 대의 GPS 위치정보를 활용해 5분 단위로 구간 속도를 수집하며, 도시고속도로는 별도 검지기로 수집합니다. TOPIS는 이렇게 수집된 속도 정보를 5분, 15분, 30분, 1시간 단위 통계로 가공해서 제공.

1) 서울시 교통량 이력 정보: <https://data.seoul.go.kr/dataList/OA-13316/A/1/datasetView.do>

시간(Hour) 단위, 요청 시 특정 연/월/일과 시간(0~23시) 값을 인자로 전달하여 데이터를 조회하는 구조 500m×500m 같은 격자(Grid) 단위가 아니라, 서울시 주요 도로 상에 설치된 '교통량 관측 지점(지점번호)' 단위를 기준으로 수집

2) 서울시 실시간 도로 소통 정보: <https://data.seoul.go.kr/dataList/OA-13291/A/1/datasetView.do>

실시간 (약 1분 ~ 5분 간격)으로 서울시 주요 도로의 속도 및 소통 상태가 갱신. 격자 단위가 아닌 '도로 링크(Link) 및 노드(Node)' 단위로 수집. 해당 링크 구간 내의 평균 속도, 소통 상태(원활, 서행, 정체) 등의 정보가 제공.

3) 서울 교통 빅데이터 플랫폼: <https://t-data.seoul.go.kr/userguide/guideopenapi.do>

일(Day) 단위, 시간(Hour) 단위 집계 데이터 또는 실시간 연계 데이터가 주기별로 업데이트. 500m×500m 같은 임의의 정형 격자 단위보다는 행정동 단위, 대중교통 정류소/노선 단위, 혹은 도로 링크 단위 등 교통 체계의 지리적 특성(GIS 표준 노드-링크 체계)에 맞추어 공간 단위가 구성되어 있음.

- **문제점:** 강수량은 지표면 전체를 균일한 정사각형 격자로 나눠 값을 주는 래스터(raster) 데이터, 교통정보는 실제 도로를 따라 이어지는 선(링크·구간) 단위 벡터(vector) 데이터. 즉 "이 위경도 500m×500m 칸의 교통량"이 아니라 "이 도로 구간(예: 강남대로 A교차로~B교차로)의 속도"라는 형태로 제공]

- **실무 대안: 링크데이터를 500m 격자로 변환->교통정보 API를 그대로 쓰려면 후처리(공간 매핑) 필요**

1. 각 도로 링크에는 시점·종점 좌표가 있으므로, 그 좌표들이 어느 500m 격자 안에 들어가는지 계산
2. 하나의 격자 안에 여러 링크가 걸쳐 있으면, 그 링크들의 속도를 평균(또는 길이로 가중평균)해서 그 격자의 대표 교통값으로 만듭니다.
3. 도로가 아예 지나가지 않는 격자는 교통 채널에서 결측(NaN) 처리하고, 위험도 계산 시 해당 채널은 자동으로 제외(0으로 처리)하면 됩니다 — 지난번 만들어드린 코드의 np.nan 처리 로직과 자연스럽게 맞물립니다.

(Tip: 아래 24~72시간 사후 모니터링 시스템 코드에서 **fetch_traffic_grid** 함수 안에 "링크→격자 변환" 로직을 넣을 수 있다함. TOPIS API 응답 형식(링크ID, 시점/종점 좌표, 속도)을 기준으로 격자별 평균값을 계산하는 코드를 짜는 거라 함. 이는 실제 API 받아본 후 수정)

3. 지하수위 API

- 1) 서울 열린데이터광장 "서울시 보조지하수 관측망 관측정보"(OA-15611): 서울시 25개 구청에서 관리하는 보조지하수 관측망 관측정보를 Open API로 제공합니다.

- 보조지하수측정망 운영: <https://www.gims.go.kr/assilnstallMthd.do>

- 서울보조지하수 관측 결과: <https://data.seoul.go.kr/dataList/OA-15611/S/1/datasetView.do?tab=A>

- 2) 국가지하수정보센터(GIMS)의 국가지하수측정망: 자동관측장비를 통해 지하수위를 지속적으로 관측. 공공데이터포털에 지하수의 수위·수질 변동실태를 조회할 수 있는 시간 단위 즉, 1시간마다 업데이트되는 Open API가 있다(한국수자원공사 제공).

*국가지하수정보센터(GIMS): <https://www.gims.go.kr/assilnstallMthd.do>

*한국수자원공사_국가지하수측정자료조회서비스(시간자료)

: <https://www.data.go.kr/data/15114511/openapi.do?recommendDataYn=Y>

4. 서울시 하수관 노후도 · 누수데이터

2023년 12월 기준 서울 하수관로 총연장 1만866km 중 50년 이상 된 관로가 3300km(30.4%)이며, 30년이 넘은 관로는 55.5%에 달합니다. 이 자치구별 노후 비율(진선미 의원실이 서울시로부터 받은 자료)을 1단계 기준 자치구별 가중치로 쓰면 됩니다.

*노후관 데이터 활용 이유: 최근 지반침하 원인을 살펴보면 하수관 손상이 전체의 46.2%를 차지하며 가장 많았고, 상수관·기타 매설물 손상까지 합하면 지하시설물 관련 원인이 전체의 63.2%에 달한다. 국토교통부 자료 기준으로도 최근 5년간(2020~2024) 전국 지반침하 867건 중 45.4%(394건)가 하수관 손상 때문이었다. → 하수관 노후도는 원인 비중 1위이므로 가중치를 가장 높게 줄 근거로 명확.

- 상수도 GIS 및 3차원 공간정보시스템: <https://www.seoulsolution.kr/ko/content/1320>

- 서울특별시_풍수해 침수예상도 공간정보: <https://www.data.go.kr/data/15125748/fileData.do>

- **상수도관:** 서울시는 '상수도지리정보시스템'을 도입해 상수도 관로 위치를 정확히 관리하고 있으며, 2015년부터 ICT를 활용한 상시누수진단시스템과 상수도관망 정밀안전진단을 실시하고 있습니다. 개별 배관 하나하나의 재질·매설연도·누수이력이 이 시스템 안에 존재합니다. 서울시가 정의하는 '노후관'은 회주철관, 아연도강관, 비내식성관으로 누수가 잦거나 구조적 강도가 저하된 관, 관 내부에 녹이 발생해 녹물이 많이 나오는 관 등을 말합니다. — 즉 "노후"를 나이뿐 아니라 재질·부식 상태로도 정의하는 정교한 기준이 이미 있습니다. 앞으로 '상수도관 노후 평가관리프로그램'을 추가 개발할 예정이며, 2022년까지 총 3,924km의 상수도관로 및 부속시설물을 정확히 조사·탐사·측량해 과학적 의사결정 기초자료로 활용할 계획이라고 밝혔습니다.
- **하수관로:** 서울시는 30년 이상 경과한 하수관로를 대상으로 조사와 상태평가를 실시하고, 정비가 필요한 구간을 우선순위에 따라 정비하고 있으며, AI·CCTV·라이다·GIS 등 디지털 기술을 도입해 과학적 유지관리 체계를 고도화하고 있습니다. CCTV·라이다로 관 내부 손상 상태까지 개별 구간별로 조사하고 있다는 뜻입니다.

▶ **문제점:** 격자 지도는 공개하지 않음.

▶ **현실적으로 쓸 수 있는 대안**

방법	상세
자치구 단위 집계 통계	지난번 찾아드린 것처럼 자치구별 노후관 비율(30/50년 이상 %)은 언론보도·국정감사 자료로 확인 가능 — 격자보다 거친 해상도지만 유일한 공개 경로
정보공개청구	서울시 정보소통광장(opengov.seoul.go.kr)에 "○○구 하수관로 GIS shapefile(관로별 매설연도 포함)"을 구체적으로 특정해 청구하면, 개인정보·보안 문제가 없는 선에서 원본 GIS 데이터를 받을 가능성이 있습니다. 액상화 지도 원문 보고서처럼, 시가 이미 구축한 데이터라 "만들어달라"가 아니라 "이미 있는 걸 달라"는 요청이라 성사 가능성이 상대적으로 높습니다
하수관로 결함(CCTV 조사) 통계	환경부·한국환경공단이 자치구별 "하수관로 정밀조사 결과 결함등급 비율" 같은 요약 통계를 낼 때가 있어, 이걸로 상대적 위험도 근사치를 만들 수 있습니다
간접 프로кси로 대체	배관 자체 데이터가 없다면, 지난번 액상화 지도처럼 "매립토/충적토 지역(느슨하고 물을 많이 머금은 지반)"과 "택지개발 연도"(오래된 동네일수록 배관도 오래됐을 가능성)를 결합해 간접 추정하는 방법도 있습니다 — 다만 이걸 근사치일 뿐 실측이 아니라는 점을 문서에 명시하셔야 합니다

5. 서울시 지반액상화 위험지도(2022) - 클로드에 보고서 내용 정리해둔거 보기

https://openlab.eseoul.go.kr/gallery/viewStoryMapContent.do?scid=YWcf0_tmkaVUouTMjx9oLQ

: 한국건설기술연구원이 개발한 기술로 광역단위 액상화 위험도 평가 및 가시화 기술을 활용해 서울시 액상화 위험지도를 구축했고, **지하수 수위정보, 지반정보, 수치표고모델정보 등을 활용**해 서울 전역을 대상으로 만들어짐. 이는 지진 시 액상화를 전제로 만든 지도지만, 지하수위·지층 특성이 반영돼 있어 "어느 지역 지반이 원래 무른가"를 보는 데 유용. **국가 표준 지진 기준 + 실측 지반자료 + 검증된 국책 연구 프로그램 + 국제적으로 공인된 지수**를 결합한 정석적인 구성이기 때문. 500/1000/2400년 = 지진 강도를 확률(재현주기)로 표현한 것, 누적 개념 아님. 숫자가 클수록 더 드물고 강한 지진. 보고서에는 250m×250m로 되어있으나 실제 서울시에서 액상화 지도 업로드 시 격자 변경 가능성 있음.

***한계점:** "광역 스크리닝용"이라는 태생적 한계가 있음. 이걸 지진으로 인한 액상화 위험을 보여주는 지도이지, 지난번 말씀하신 폭우·노후하수관 기반 싱크홀 위험과는 발생 메커니즘이 다릅니다. (같은 "지반이 약하다"는 배경정보로는 참고할 수 있지만, 원인이 다름)

***액상화 위험지도 보고서 전문:** "Tech-lead형 액상화 위험지도 구축기술 고도화 연구", KISTI ScienceON, 관리번호 TRKO202300003314 (<file:///C:/Users/hyeli/Downloads/OTKCRK230207.pdf> – 링크 복붙하면 파일 바로 뜸)

6. 그 외 지반 위험도 판별용 정보들

: 다현언니가 메일로 보내준 자료들

<실시간 교통량, 서울시 노후관, 24~72시간 사후 모니터링 시스템 활용 이유>

2013년 시카고 도심 사례를 보면, **전날 내린 폭우로 주택가 도로**에 폭 12m 규모의 지반함몰이 발생해 차량 3대가 매몰되었는데, 붕괴는 강우로 지반이 느슨해지면서 1915년에 설치된 **낮은 상하수도관에 균열**이 발생하고, 그 결과 강한 압력으로 하부 토사가 유실되며 지하공동이 형성되어 일어났습니다.

=> 교통이 혼잡X / 정체로 인해 반복적인 가속·제동·정지 하중이 걸림O

=> 전날 내린 비로 인해 다음날 사건 발생

<24~72시간 사후 모니터링 시스템 설계>

위험도를 비가 그치는 순간 즉시 리셋이 아닌, "감쇠(decay) 함수" 적용

- **1단계[0~24시간]:** 유지구간(plateau) - 비가 그쳐도 위험도를 그대로 유지. 이 구간에는 감쇠를 적용하지 않음. 이미 침투한 물이 지반 내부에서 계속 작용 중이기 때문.
- **2단계[24~72시간]:** 지수감쇠구간* - 이 구간부터 위험도가 서서히 1단계 수준으로 돌아감.

*지수감쇠(exponential decay)함수 쓰는 이유: 실제 수문학(선행강우지수, API)에서 쓰는 표준 방식이고, 재귀적으로 계산할 수 있어서 전체 이력을 저장할 필요 없이 "직전 값 하나"만 있으면 되기 때문에 실시간 시스템 구현에 훨씬 유리하다.

[설계 구조]

1. 격자 생성 (build_seoul_grid)

: 서울시 경계 bounding box(위경도)를 500mX500m의 격자 중심점 배열*을 만든다. 테스트 결과 약 5,032개 격자가 생성됐다. 이걸 사각형 경계 기준이라 실제로는 서울시 행정경계 shapefile과 겹쳐서 시영역 밖 격자를 제거하는 후처리가 필요하다 — 이 부분은 지오판다스(geopandas)로 쉽게 처리 가능

*배열 활용 이유: 서울시 면적 약 605km² ÷ 0.25km²(500m×500m) ≈ 2,420개 격자. 이를 격자 수만큼 파이썬 객체로 수만 개 생성하는 건 비효율적. 이 정도 규모부터는 개별 객체 방식보다 numpy 배열로 한 번에 벡터 연산하는 방식이 실전에서 훨씬 빠르고 코드도 깔끔 (자료구조에서 Numpy배열 배움)

2. 공통 감쇠함수 (decay_factor_array)

강수량·지하수위·교통량 세 채널을 전부 활용하는 하나의 함수를 만든다. 즉 채널별 격자별 별도의 함수를 만들지 않는다. 이 함수 하나를 공유한다. 채널마다 별도 함수를 만들지 않았다.

3. GridRiskField 클래스

격자당 객체를 만드는 대신, 격자 수(N) 길이의 numpy 배열 세트로 전체 상태를 관리한다. update_channel 메서드 하나가 세 채널(rain/groundwater/traffic) 전부에 재사용된다 — 채널 이름만 바꿔서 호출하면 된다.

4. API 연동 지점

fetch_rainfall_grid, fetch_groundwater_grid, fetch_traffic_grid 세 함수는 지금 더미(랜덤값)를 반환하도록 되어 있다. 실제 기상청 API허브·GIMS·TOPIS 연동 시 이 세 함수의 내부 코드만 실제 API 호출로 교체하면 나머지 로직(감쇠·단계판정·실시간루프)은 전혀 손댈 필요가 없다.

5. 실시간 루프 (run_realtime_loop)

채널마다 실제 API 갱신주기가 다르다는 걸 반영했다 — 강수량·교통량은 5분, 지하수위는 1시간 주기로 각각 독립적으로 폴링한다. 메인 루프는 짧은 간격(tick_sec)으로 깨어나서 "지금 갱신할 때가 된 채널"만 호출하는 방식이라, 불필요한 API 호출 없이 상시 구동이 가능하다. max_iterations=None으로 두면 무한 루프로 계속 돈다(실제 운영용).

-실제 활용 시 수정할 것-

- 3단계 판정 조건을 (rain_f > 0) & ((gw_f > 0) | (tf_f > 0))로 짜서, 위에서 계획했던 대로 "강수+지하수위" 또는 "강수+교통이상" 둘 중 하나만 겹쳐도 3단계가 뜨도록 넣어뒀다. 이 조건식은 얼마든지 수정 가능.
- 배경위험(baseline_scores) 즉, 1단계 위험도는 액상화 등급+노후관 비율로 미리 계산해서 배열로 로드하는 부분만 채우면 된다. 현재는 랜덤값으로 채워져 있다.
- 실제 운영 시에는 time.sleep 기반 루프 대신 별도 프로세스/스케줄러(cron, APScheduler)로 돌리거나, 웹 서버와 함께 백그라운드 스레드로 구동하는 형태가 안정적이다.
- 위에 5번에서 얘기한 실시간 루프가 현재는 테스트용으로 5회만 돌게 해둔 상태. 무한 루프로 수정.

* 파이썬 코드는 맨 마지막 [부록]에 있음.

[이공계 학술제] 목적 및 심사 기준에 따른 보완할 점

- **대외 목적:** 공모전을 준비하면서 다양한 공학적 접근을 통한 공학적 사고력 증진하는 것
- **심사기준:** 창의성, 기술적 완성도, 실용성, 이 연구가 왜 얼마나 필요한건지, 자신의 전공(정보통신공학)을 살렸는지 등

<부족한 점>

- **창의성:** 지금 만드시려는 건 "공공 API 4개를 가져다 가중합산하고 지도에 색칠하는" 구조입니다. 사실 이미 존재하는 것들과 거의 겹칩니다 — 서울시 지반침하 위험지도(서울연구원), 액상화 위험지도(오늘 보신 그 지도), 서울 안전누리 GPR 탐사지도. "기존 데이터를 처음으로 하나로 합쳤다"는 점은 차별점이지만, 심사위원 입장에선 "그래서 뭘 새로 만든 거지?"라는 질문이 바로 나올 겁니다.
- **기술적 완성도:** if-then 임계값 규칙(비가 60mm 넘으면 2단계) 정도는 사실 알고리즘이라 부르기 애매합니다. 데이터 파이프라인(API 호출→가중합→시각화) 자체는 기술이지만, 정보통신공학 전공생이 심사위원 앞에서 "이게 제 전공 역량입니다"라고 내세우기엔 알습니다.
- **전공 활용도** — 이게 가장 치명적입니다. 지금 하시는 작업은 사실 GIS/도시공학/데이터분석에 훨씬 가깝습니다. "정보통신공학"이 어디 들어가 있는지 심사위원이 찾기 어려울 거예요. 통신(네트워크), 신호처리, 임베디드, IoT, 무선통신 프로토콜 같은 요소가 지금 계획엔 전혀 없습니다.
- **실용성 / 연구 필요성:** 이 부분은 오히려 강점입니다. 지금까지 대화하시면서 "실시간으로 결합된 위험지도가 현재 없다"는 걸 실제 자료 조사로 입증하셨거든요 — 이건 "왜 이 연구가 필요한가"에 대한 좋은 근거가 됩니다.

<보완 방향>

우선순 위	항목	이유
1순위	IoT 센서 프로토타입 (2번)	전공 어필 가장 강력, 실물 시연 가능
2순위	이상탐지 알고리즘 (3단계 판정 로직 고도 화)	기술적 완성도 직결, 코드로 구현 가 능
3순위	알림 전달 아키텍처 설계(문서화만)	시간 적게 들면서 전공 연계 어필
4순위	실시간 파이프라인 아키텍처(문서화만)	있으면 좋지만 필수는 아님

★ 보완할점 - 추가내용(아래 내용이 더 좋은 듯)

1. 이중 해상도 데이터 통합에 칼만 필터/베이저안 센서 퓨전 적용 — 가장 추천

지금 문제가 뭐였냐면: 액상화(250m), 노후관(자치구), 강수량(500m), 지하수위(관측정)가 해상도도 다르고 신뢰도(측정 오차)도 다릅니다. 지금까지는 "그냥 리샘플링해서 합친다"는 수준이었는데, 이걸 다중 센서 퓨전(Multi-sensor Data Fusion) 문제로 재정의하면 완전히 달라집니다.

각 데이터 소스마다 "신뢰도(분산)"를 다르게 부여합니다. 예: 액상화지도는 학술검증된 정적 데이터라 분산이 작고(신뢰도 높음), 지하수위는 관측정이 멀리 떨어져 있어 보간 오차가 크다(분산 큼)

칼만 필터(Kalman Filter) 또는 베이저안 업데이트로 이 서로 다른 신뢰도의 값들을 통계적으로 결합해서 격자별 최종 위험도와 그 "불확실성(신뢰구간)"까지 같이 산출

이건 통신공학에서 배우는 추정이론(estimation theory)·신호처리와 정확히 같은 수학입니다 — GPS 위치추정, 무선채널 추정에 쓰이는 바로 그 기법을 지반 위험도 추정에 적용하는 것

심사위원에게 어필 포인트: "단순 가중합이 아니라, 각 데이터의 측정 불확실성을 정량화해서 최적 추정을 낸다"는 설명 한 줄이면 전공 역량이 확 삽니다. 파이썬으로 오늘 안에 구현 가능한 수준입니다 (scipy나 직접 구현).

2. 지하수위 이상탐지를 규칙기반 → 예측모델 기반으로

"1σ 넘으면 3단계" 대신, **시계열 예측 모델(간단한 칼만필터 기반 상태공간모델, 혹은 경량 LSTM)**로 "다음 시점 지하수위가 이 정도일 것"을 예측하고, 실측값과의 잔차(residual)가 크면 이상으로 판정하는 구조로 바꾸세요.

이건 통신 시스템에서 채널 상태를 예측하고 이상을 탐지하는 것과 동일한 프레임워크입니다

코드 몇십 줄로 구현 가능하고, "예측 대비 잔차 기반 이상탐지"는 논문에서도 자주 쓰는 정당한 방법론이라 근거도 탄탄합니다

3. 알림 전파를 통신 네트워크 문제로 설계 (구현 안 해도 문서로 충분)

3단계 경보가 났을 때 "누구에게 어떻게 빨리 알릴 것인가"를 통신공학적으로 풀어보세요.

지오펀싱 기반 메시지 브로드캐스트: 위험 격자 반경 내 사용자에게만 선택적으로 알림 전송하는 구조 설계

MQTT Pub/Sub 구조로 위험도 변경 이벤트를 실시간 구독자들에게 전파하는 아키텍처(QoS 레벨 선택 근거까지 — 재난 알림이니 QoS 2로 반드시 전달 보장해야 한다는 식)

지역 커버리지가 넓은 재난문자(CBS)와 앱 푸시(FCM) 중 상황별로 어떤 걸 쓸지 지연시간·도달률 비교 표로 정리

이건 실제 서버 없이 아키텍처 다이어그램 + 설계 근거만으로도 충분히 심사에서 "정보통신공학 전공을 살렸다"는 인상을 줍니다.

4. (여유 되면) 무선 센서 네트워크 배치를 "시뮬레이션"으로 설계

실제 센서를 안 사더라도, "만약 이 시스템을 실제 배치한다면"이라는 가정 하에:

LoRa/NB-IoT 중 어떤 통신 방식이 지하 매설 환경에 적합한지 링크버지(link budget) 계산으로 비교
서울 특정 자치구를 대상으로 센서 노드 커버리지 시뮬레이션(전파 감쇠 모델 적용)

이건 하드웨어 없이 "설계 역량"만으로 통신공학 색채를 강하게 낼 수 있는 방법입니다. 다만 오늘 마감하시면 우선순위는 낮게 두시는 게 좋겠어요.

여담) - 보고서 작성 계획 -

즉, 그저 공공데이터를 연결해둔 앱 하나 만들기. 이런 느낌을 줄 수 있기 때문에 공공데이터를 그대로 갖다 쓴 게 아니라, 신호처리·추정이론을 적용해 통계적으로 통합했다는 내용을 담을 수 있는 활동 추가하는게 좋을 듯.

그렇게 되면 전공 수업에서 배운 내용을 응용하여 적용해보면서 공학적 설계역량을 기를 수 있었다는 점 어필 가능. 또한 각각의 여러 공공데이터를 하나의 통합 데이터로 본점에서 공학적 역량과 창의성에서 좋은 점수 받은 가능성 있음. 그리고 기존에 존재하는 단순 측정 시스템보다 24~72시간 경과 지켜보고 감쇠함수를 만들어 실시간 적용하는게 독창적이게 보일 수 있음. 실제 구현하는 과정 속에서 오차 범위나 정확도나 성공률 퍼센트로 자세하게 기재하고 손경락교수님의 지난 1학기 iot팀플 평가표에 프로젝트 평가 방식은 어떤식으로 되는지 상세히 나와있으니 이를 참고하여 보고서 작성하기.

-> 중요: 다중 데이터 융합, 최대 72시간 위험도 반영 및 감쇠함수 설정, 신호처리, 이상탐지 알고리즘, 추정이론, 시뮬레이션 활용 어필하기

<앞으로 해야할 것>

다현: 실시간 API를 개인 컴퓨터에 다운 및 실시간 연동 프로그램 제작, 시뮬레이션용 랜덤값 싱크홀 위험지도 제작

[코드]

```
# -*- coding: utf-8 -*-
```

```
"""
```

서울시 500m x 500m 격자 기반 싱크홀 위험도 실시간 산정 시스템

=====

구성

1. 서울시 경계를 500m 격자로 분할 (좌표 생성)
2. 감쇠함수(decay_factor_array) — 하나의 함수를 강수량/지하수위/교통량
3개 트리거 채널에 공통으로 재사용 (격자별 객체 생성 X, numpy 배열 기반)
3. GridRiskField — 전체 격자 상태를 배열로 관리하는 클래스
4. 실시간 루프 스켈레톤 — 각 API를 소스별 주기에 맞춰 호출하고
위험도를 갱신하는 상시 구동 루프

실제 API 연동 시에는 fetch_rainfall_grid / fetch_groundwater_grid /
fetch_traffic_grid 세 함수 내부만 실제 API 호출 코드로 교체하면 된다.

```
"""
```

```
import math
```

```
import time
```

```
import numpy as np
```

```
# =====
```

```
# 1. 서울시 500m 격자 생성
```

```
# =====
```

```
# 서울시 대략적 경계 (bounding box). 실제 프로젝트에서는 서울시 행정경계
```

```
# shapefile과 겹쳐서 시 영역 밖 격자는 제거하는 후처리를 추천한다.
```

```
SEOUL_LAT_MIN, SEOUL_LAT_MAX = 37.413, 37.715
```

```
SEOUL_LON_MIN, SEOUL_LON_MAX = 126.764, 127.184
```

```
GRID_SIZE_M = 500.0 # 격자 한 변 길이 (m)
```

```
DEG_LAT_PER_M = 1 / 111_000 # 위도 1도 ≈ 111km
```

```
# 경도 1도 거리는 위도에 따라 달라지므로 서울 평균위도(약 37.55도) 기준으로 보정
```

```
DEG_LON_PER_M = 1 / (111_000 * math.cos(math.radians(37.55)))
```

```
def build_seoul_grid(lat_min=SEOUL_LAT_MIN, lat_max=SEOUL_LAT_MAX,  
                    lon_min=SEOUL_LON_MIN, lon_max=SEOUL_LON_MAX,  
                    grid_size_m=GRID_SIZE_M):
```

```
    """
```

```
    서울시 경계 bounding box를 500m 간격 격자 중심점(위경도) 배열로 변환.
```

```
    반환: (grid_lat, grid_lon) — 각각 길이 N인 numpy 배열
```

```
    """
```

```
    lat_step = grid_size_m * DEG_LAT_PER_M
```

```
    lon_step = grid_size_m * DEG_LON_PER_M
```

```
    lats = np.arange(lat_min, lat_max, lat_step)
```

```
    lons = np.arange(lon_min, lon_max, lon_step)
```

```
    lon_grid, lat_grid = np.meshgrid(lons, lats)
```

```
    grid_lat = lat_grid.ravel()
```

```
    grid_lon = lon_grid.ravel()
```

```
    return grid_lat, grid_lon
```

```
# =====
```

```
# 2. 공통 감쇠함수 (모든 트리거 채널이 동일 함수를 재사용)
```

```
# =====
```

```
def compute_lambda(plateau_hours: float, cutoff_hours: float,
```

```
                  residual_ratio: float = 0.05) -> float:
```

```
    """
```

```
    cutoff_hours 시점에 잔존비율(residual_ratio)만 남도록 감쇠계수(lambda) 역산.
```

```
    """
```

```
    decay_span = cutoff_hours - plateau_hours
```

```
return -math.log(residual_ratio) / decay_span
```

```
def decay_factor_array(trigger_time: np.ndarray, now_ts: float,
                      plateau_hours: float, cutoff_hours: float,
                      lam: float) -> np.ndarray:
    """
    격자 N개에 대한 감쇠계수를 한 번에 계산하는 범용 함수.
    강수량/지하수위/교통량 트리거 모두 이 함수 하나를 공유한다.

    trigger_time : 길이 N 배열, 각 격자의 마지막 트리거 시각(epoch seconds).
                  트리거가 없던 격자는 np.nan.
    now_ts       : 현재 시각(epoch seconds)
    반환        : 길이 N 배열, 0~1 사이의 감쇠계수
    """

    factor = np.zeros(trigger_time.shape[0], dtype=np.float64)
    valid = ~np.isnan(trigger_time)
    if not np.any(valid):
        return factor

    elapsed_h = (now_ts - trigger_time[valid]) / 3600.0
    f = np.zeros(elapsed_h.shape, dtype=np.float64)

    plateau_mask = elapsed_h <= plateau_hours
    decay_mask = (elapsed_h > plateau_hours) & (elapsed_h <= cutoff_hours)
    # cutoff_hours 초과분은 0으로 이미 초기화되어 있음

    f[plateau_mask] = 1.0
    f[decay_mask] = np.exp(-lam * (elapsed_h[decay_mask] - plateau_hours))

    factor[valid] = f
    return factor

# =====
# 3. 전체 격자 상태를 배열로 관리하는 위험도 필드
# =====
class GridRiskField:
    """
    격자별 객체를 생성하지 않고, N개 격자의 상태를 numpy 배열 세트 하나로
    관리한다. 트리거 채널은 강수량 / 지하수위 / 교통량 3종.
    """

    def __init__(self, grid_lat, grid_lon, baseline_scores,
                 plateau_hours=24.0, cutoff_hours=72.0, residual_ratio=0.05):
        self.lat = np.asarray(grid_lat, dtype=np.float64)
        self.lon = np.asarray(grid_lon, dtype=np.float64)
        self.n = len(self.lat)

        assert len(baseline_scores) == self.n, "baseline_scores 길이가 격자 수와 다릅니다."
        self.baseline = np.asarray(baseline_scores, dtype=np.float64)

        self.plateau_hours = plateau_hours
        self.cutoff_hours = cutoff_hours
        self.lam = compute_lambda(plateau_hours, cutoff_hours, residual_ratio)

        # 트리거 채널별 상태 (모두 길이 N 배열)
        self.trigger_time = {
            "rain": np.full(self.n, np.nan),
            "groundwater": np.full(self.n, np.nan),
            "traffic": np.full(self.n, np.nan),
        }
        self.peak = {
            "rain": np.zeros(self.n),
            "groundwater": np.zeros(self.n),
        }
```

```

        "traffic": np.zeros(self.n),
    }

    # 각 채널을 마지막으로 언제 조회(fetch)했는지 기록 -> 주기 제어용
    self.last_fetch_ts = {"rain": 0.0, "groundwater": 0.0, "traffic": 0.0}

# -----
# 트리거 갱신: 세 채널 모두 동일한 로직 -> 공통 메서드 하나로 처리
# -----
def update_channel(self, channel: str, grid_scores, now_ts: float, threshold: float):
    """
    channel      : "rain" | "groundwater" | "traffic"
    grid_scores  : 이번 주기 격자별 관측/환산 점수 (길이 N)
    threshold    : 트리거 발동 기준점수
    """
    grid_scores = np.asarray(grid_scores, dtype=np.float64)
    trig_t = self.trigger_time[channel]
    peak = self.peak[channel]

    hit = grid_scores >= threshold
    should_update = hit & ((grid_scores >= peak) | np.isnan(trig_t))

    peak[should_update] = grid_scores[should_update]
    trig_t[should_update] = now_ts
    self.last_fetch_ts[channel] = now_ts

# -----
# 최종 위험도 / 단계 조회 (전체 격자 벡터 연산 한 번)
# -----
def current_state(self, now_ts: float):
    factors = {
        ch: decay_factor_array(self.trigger_time[ch], now_ts,
                               self.plateau_hours, self.cutoff_hours, self.lam)
        for ch in ("rain", "groundwater", "traffic")
    }

    risk_by_channel = {
        ch: self.baseline + (self.peak[ch] - self.baseline) * factors[ch]
        for ch in factors
    }
    # 세 채널 중 가장 위험한 값을 최종 위험도로 채택
    final_risk = np.maximum.reduce(list(risk_by_channel.values()))

    rain_f = factors["rain"]
    gw_f = factors["groundwater"]
    tf_f = factors["traffic"]

    stage = np.ones(self.n, dtype=int)
    stage[rain_f > 0] = 2
    stage[gw_f > 0] = 2
    stage[tf_f > 0] = 2
    # 기본 1단계 (배경위험)
    # 강수 트리거 유효 -> 2단계
    # 3단계: 강수 + (지하수위 급변 또는 교통 이상정체) 동시 유효
    stage[(rain_f > 0) & ((gw_f > 0) | (tf_f > 0))] = 3

    return final_risk, stage, factors

# =====
# 4. 외부 API 연동 지점 (실제 구현 시 이 함수들만 교체)
# =====
def fetch_rainfall_grid(grid_lat, grid_lon) -> np.ndarray:
    """
    TODO: 기상청 API허브(초단기실황/500m 격자분석데이터) 호출로 교체.
    반환값은 격자별 "3시간 누적강수 환산 위험점수" (0~100) 배열이어야 한다.
    지금은 시뮬레이션을 위한 더미 데이터를 반환한다.
    """
    return np.random.uniform(0, 20, size=len(grid_lat))

```

```
def fetch_groundwater_grid(grid_lat, grid_lon) -> np.ndarray:
    """
    TODO: 국가지하수정보센터(GIMS)/서울시 보조지하수 관측망 API 호출로 교체.
    관측정 값을 최근접 보간하여 격자별 "평년 대비 이상편차(sigma 배수)"로
    변환한 값을 반환해야 한다. 지금은 더미 데이터.
    """
    return np.random.uniform(0, 1.5, size=len(grid_lat))
```

```
def fetch_traffic_grid(grid_lat, grid_lon) -> np.ndarray:
    """
    TODO: 서울시 TOPIS 실시간 도로소통정보 API 호출로 교체.
    도로 링크 속도를 평시 대비 저하율로 환산한 격자별 "정체 이상도" 점수.
    지금은 더미 데이터.
    """
    return np.random.uniform(0, 1.0, size=len(grid_lat))
```

```
# =====
# 5. 실시간 상시 구동 루프
# =====
# 각 데이터 소스마다 실제 API 갱신 주기가 다르므로, 채널별로 독립적인
# 폴링 주기를 두고 메인 루프에서 tick마다 "이번에 갱신할 때가 된 채널"만
# 호출하는 방식으로 구현한다.
```

```
FETCH_INTERVAL_SEC = {
    "rain": 5 * 60,          # 기상청: 5분 주기
    "groundwater": 60 * 60, # 지하수위: 1시간 주기 (API 특성상 시간 단위 갱신)
    "traffic": 5 * 60,       # TOPIS 실시간 소통정보: 5분 주기
}
```

```
TRIGGER_THRESHOLD = {
    "rain": 60.0,            # 예: 3시간 60mm(호우주의보 기준) 환산 점수
    "groundwater": 1.0,     # 예: 평년 대비 1 표준편차(sigma) 이상 변동
    "traffic": 0.6,         # 예: 평시 대비 정체(속도저하) 이상도 0.6 이상
}
```

```
STATE_LOG_INTERVAL_SEC = 60 # 위험도 요약 로그 출력 주기
```

```
def run_realtime_loop(field: GridRiskField, tick_sec: int = 30, max_iterations=None):
    """
```

```
    상시 구동되는 메인 루프.
    tick_sec 간격으로 깨어나서, 각 채널의 갱신 주기가 도래했는지 확인 후
    필요한 채널만 fetch -> update_channel 호출. 이후 전체 격자 위험도를
    재계산해 요약 로그를 출력한다.
```

```
    max_iterations: 테스트용. None이면 무한 루프(실제 운영용).
    """
```

```
    fetchers = {
        "rain": fetch_rainfall_grid,
        "groundwater": fetch_groundwater_grid,
        "traffic": fetch_traffic_grid,
    }
```

```
    last_log_ts = 0.0
    it = 0
    while True:
        now_ts = time.time()
```

```
        for channel, interval in FETCH_INTERVAL_SEC.items():
            if now_ts - field.last_fetch_ts[channel] >= interval:
```

```

        scores = fetchers[channel](field.lat, field.lon)
        field.update_channel(channel, scores, now_ts,
                             threshold=TRIGGER_THRESHOLD[channel])

    if now_ts - last_log_ts >= STATE_LOG_INTERVAL_SEC:
        risk, stage, _ = field.current_state(now_ts)
        n1 = int(np.sum(stage == 1))
        n2 = int(np.sum(stage == 2))
        n3 = int(np.sum(stage == 3))
        print(f"[{time.strftime('%Y-%m-%d %H:%M:%S')}] "
              f"격자 {field.n}개 | 1단계 {n1} / 2단계 {n2} / 3단계 {n3} | "
              f"최대위험도 {risk.max():.1f}")
        last_log_ts = now_ts

    it += 1
    if max_iterations is not None and it >= max_iterations:
        break
    time.sleep(tick_sec)

# =====
# 6. 실행 예시
# =====
if __name__ == "__main__":
    grid_lat, grid_lon = build_seoul_grid()
    n_grids = len(grid_lat)
    print(f"생성된 서울시 500m 격자 수: {n_grids}")

    # 정적 배경위험(1단계) — 액상화 등급 + 하수관로 노후도 비율 기반으로
    # 사전에 계산해 둔 배열을 여기에 로드한다. (지금은 예시로 랜덤값 사용)
    baseline_scores = np.random.uniform(5, 40, size=n_grids)

    field = GridRiskField(grid_lat, grid_lon, baseline_scores,
                           plateau_hours=24.0, cutoff_hours=72.0, residual_ratio=0.05)

    # 실제 운영 시: run_realtime_loop(field, tick_sec=30) # 무한 루프로 상시 구동
    # 테스트 실행 (5회 tick만 확인):
    run_realtime_loop(field, tick_sec=1, max_iterations=5)

```